

Detecting Windows Attacks with Splunk Module

Detecting Common User/Domain Recon

Domain Recon

AD domain reconnaissance represents a pivotal stage in the cyberattack lifecycle. During this phase, an adversary will attempt to gather information about the target environments, map out domain structure and identify security measures/potential vulnerabilities. While conducting AD recon, attackers focus on identifying critical components such as domain controllers, user accounts, trust relationships, groups, OUs and GPOs among others.

An example of AD domain reconnaissance is when an adversary utilises the `net.exe` executable to obtain a list of Domain Administrator accounts.

```
net group "Domain Admins" /domain
```

Other tools/commands we may see being used for domain recon include:

```
whoami /all
```

```
wmic computersystem get domain
```

```
net user /domain
```

```
arp -a
```

```
nltest /domain_trusts
```

To detect these, we can employ PowerShell command-line monitoring. Attackers may also use BloodHound to map out potential attack paths in a network by using graph theory and relationship mapping to show privilege escalation routes. SharpHound is a c# data collector for BloodHound. An example might be an adversary running sharpHound with all collection methods to gather data they can import into BloodHound:

```
.\Sharphound3.exe -c all
```

Under the hood, the BloodHound collector executes numerous LDAP queries directed at the DC to gather information. Detecting this can be difficult since windows event log does not record these queries. Windows recommends using event 1644 which are the LDAP performance monitoring logs however this still is not a perfect solution. A better approach is to use the `Microsoft-Windows-LDAP-Client` ETW provider. In addition to being able to use YARA to search these LDAP queries, the Microsoft APT team has compiled a list of LDAP filters commonly used by recon tools.

Recon tool	Filter
enum_ad_user_comments (Metasploit)	(&(&(objectCategory=person)(objectClass=user))((description=*pass*)(comment=*pass*)))
enum_ad_computers (Metasploit)	(&(objectCategory=computer) (operatingSystem=*server*))
enum_ad_groups (Metasploit)	(&(objectClass=group))
enum_ad_managedby_groups (Metasploit)	(&(objectClass=group)(managedBy=*)), (&(objectClass=group)(managedBy=*) (groupType:1.2.840.113556.1.4.803:=2147483648))
Get-NetComputer (PowerView)	(&(sAMAccountType=805306369)(dnshostname=*))
Get-NetUser - Users (Powerview)	(&(samAccountType=805306368) (samAccountName=*))
Get-NetUser - SPNs (Powerview)	(&(samAccountType=805306368) (servicePrincipalName=*))
Get-DFSshareV2 (Powerview)	(&(objectClass=msDFS-Linkv2))
Get-NetOU (PowerView)	(&(objectCategory =organizationalUnit)(name=*))
Get-DomainSearcher (Empire)	(samAccountType=805306368)

Detecting Domain Recon with Splunk

The query below can be used to identify domain recon using standard windows executables:

```

index=main source="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational"
EventID=1 earliest=1690447949 latest=1690450687
| search process_name IN
(arp.exe, chcp.com, ipconfig.exe, net.exe, net1.exe, nltest.exe, ping.exe, systeminfo.e
xe, whoami.exe) OR (process_name IN (cmd.exe, powershell.exe) AND process IN
(*arp*, *chcp*, *ipconfig*, *net*, *net1*, *nltest*, *ping*, *systeminfo*, *whoami*))
| stats values(process) as process, min(_time) as _time by parent_process,
parent_process_id, dest, user
| where mvcount(process) > 3

```

We start by filtering on the Index and the Source. The source in this case is the XML formatted Windows event log for Sysmon events. We then filter against event ID 1 which is a process creation event. We filter by time period (supposing we know when suspicious activity took place). We search for all the records where the process name is in our list of specific native executables that can be used for domain recon. We aggregate events based on the fields `parent_process`, `parent_process_id`, `dest` and `user` - for each unique combination the search calculates all unique values of the `process` field as a multi-value field named `process` as well as calculating the earliest time an event occurred in each group.

Finally we filter out all results where there are 3 or less processes executed by the parent process.

A query we could use to detect recon from BloodHound could be:

```

index=main earliest=1690195896 latest=1690285475
source="WinEventLog:SilkService-Log"
| spath input=Message
| rename XmlEventData.* as *
| table _time, ComputerName, ProcessName, ProcessId, DistinguishedName,
SearchFilter
| sort 0 _time
| search SearchFilter="*(samAccountType=805306368)*"
| stats min(_time) as _time, max(_time) as maxTime, count, values(SearchFilter)
as SearchFilter by ComputerName, ProcessName, ProcessId
| where count > 10
| convert ctime(maxTime)

```

This query filters by main index and the source that represents Windows event log data gathered by SilkETW. The `spath` command is used to extract fields from the `Message` field, which likely contains structured data such as XML or JSON. We then tidy up a bit by removing the `XmlEventData.` prefix to any field names. The `table` command is used to display the results in a tabular format using the listed fields as columns. The `sort 0 _time` command sorts the results based on the time field in ascending order where there is no limit to the number of items to

sort. The `SearchFilter` is doing the heavy lifting for detecting BloodHound as this is one of the strings that will appear in LDAP queries made by the collector. We then aggregate events based on the `ComputerName`, `ProcessName` and `ProcessID` and for each combination we calculate:

- The earliest time an event occurred
- The latest time an event occurred
- The number of events within each group
- All unique values of the `SearchFilter` field within a group.

Finally we filter by event count larger than 10 and convert the timestamps into human-readable times.

Detecting Password Spraying

Unlike traditional brute-force attacks, where an attacker tries lots of passwords for a single account, password spraying is the process of trying a smaller amount of common passwords across multiple accounts. The main goal is to gain access to any user account whilst avoiding password lockout policies. An attacker could perform a password spraying attacks from many different tools, one of which being the Spray tool:

```
sudo ./spray.sh -smb <ip> <account_list> <password_list> 100 30
```

Detecting password spraying through windows logs involves the analysis and monitoring of specific event logs to identify patterns. A key indicator we may look out for is multiple failed login attempts (event id 4625) from different user accounts but originating from the same source IP address. Other event logs that may help us are:

- 4768 and ErrorCode 0x6 - Kerberos Invalid Users
- 4768 and ErrorCode 0x12 - Kerberos Disabled Users
- 4776 and ErrorCode 0xC0000064 - NTLM Invalid Users
- 4776 and ErrorCode 0xC000006A - NTLM Wrong Password
- 4648 - Authenticate Using Explicit Credentials
- 4771 - Kerberos Pre-Auth Failed

The following query could be used to detect password spraying attempts in Splunk:

```
index=main earliest=1690280680 latest=1690289489 source="WinEventLog:Security"
EventCode=4625
| bin span=15m _time
| stats values(user) as Users, dc(user) as dc_user by src,
Source_Network_Address, dest, EventCode, Failure_Reason
```

This query searches for events that are failed login attempts and then creates buckets of 15 minutes for each event based on its `_time` field. We then use the `stats` command to aggregate events based on `src`, `Source_Network_Address`, `dest`, `EventCode` and `Failure_Reason`. For each unique combination of these fields, the search calculates the statistics:

- All unique `user` values within each group
- The count of distinct values of the `user` field (i.e how many users were affected)

Detecting Responder-like Attacks

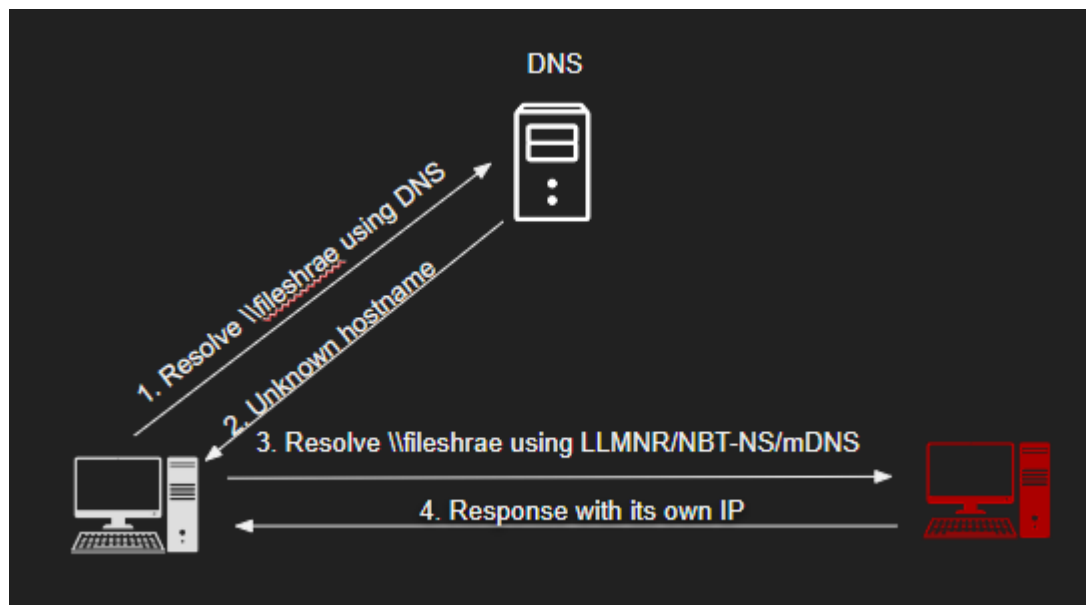
LLMNR/NTB-NS/mDNS Poisoning

LLMNR (Link-Local Multicast Name Resolution) and NBT-NS (NetBIOS Name Service) poisoning (also known as NBNS spoofing) are network-level attacks that exploit inefficiencies in name resolution protocols. Both LLMNR and NBT-NS are used to resolve hostnames to IP addresses on local networks when FQDN resolution fails. However, these protocols lack built-in security mechanisms which makes them vulnerable to spoofing and poisoning.

Typically attackers will use the Responder tool to execute these poisoning attacks. The way the attack works is this:

1. A victim device sends a name resolution query for a mistyped hostname (e.g fileshrae).
2. DNS fails to resolve the hostname.
3. The victim's device sends a name resolution query using LLMNR/NBT-NS.
4. The attackers host responds to the traffic, pretending to know the identity of the requested host and directs the victim to communicate with the attacker.

The goal of the attack is generally to get the victims machine to send across their NetNTLM hash we can then be cracked or relayed by the attacker.



Detecting LLMNR, NBT-NS and mDNS poisoning can be difficult so it is recommended to mitigate the risk of such an attack in the first place:

- Deploy network monitoring tools to detect unusual traffic
- Deploy a honeypot device that sends of misspelled DNS requests and see if we get any response.

```

$logfile = 'C:\tmpppoisoning.csv'
$requestHosts = @('CORP-TX-FILE-01','COPY-NY-DC-02') #False hostnames to request
$interval = 30 #The minimum number of seconds to wait between requests
$jitter = 30 #The maximum value for a random number of seconds to add to the
interval
while($true){
    Start-Sleep ($interval + (Get-Random ($jitter + 1)))
    try {
        $ErrorActionPreference = 'stop'
        $request = Get-Random $requestHosts
        $ipAddr = (Resolve-DnsName -LlmnrNetbiosOnly -Name
$request).IPAddress.ToString()
        $ErrorActionPreference = "continue"
        $event = [pscustomobject]@{
            date = Get-Date -format o
            host = $env:COMPUTERNAME
            request = $request
            attacker_ip = $ipAddr
            message = "LLMNR/NBT-NS spoofing by $ipAddr detected with $request
request"
        }
        Write-Output $event.message
        $event | Export-Csv -Path $logfile -Append -NoTypeInfo
    } catch [System.Management.Automation.RuntimeException],
[System.ComponentModel.Win32Exception] {
        #Suppress output of timeout errors
    } finally {
        $ErrorActionPreference = "continue"
    }
}
}

```

We can run a powershell script to automate the process of sending these false queries and we can use the `New-EventLog` powershell cmdlet to log them.

```

New-EventLog -LogName Application -Source LLMNRDetection
Write-EventLog -LogName Application -Source LLMNRDetection -EventID 19001 -
Message $msg -EntryType Warning

```

Now we can look at detecting Responder-like attacks using Splunk. Provided we have setup the new log source, we could run the following query:

```

index=main earliest=1690290078 latest=1690291207 SourceName=LLMNRDetection
| table _time, ComputerName, SourceName, Message

```


Alternatively, we could use Sysmon Event ID 22 to track DNS queries with non-existent file shares:

```
index=main earliest=1690290078 latest=1690291207 EventCode=22  
| table _time, Computer, user, Image, QueryName, QueryResults
```

We can also look for Event 4648 where attackers may use explicit logons to file shares which we can detect with the following query:

```
index=main earliest=1690290814 latest=1690291207 EventCode IN (4648)  
| table _time, EventCode, source, name, user, Target_Server_Name, Message  
| sort 0 _time
```

Detecting Kerberoasting/AS-REProasting

Kerberoasting

Kerberoasting is a technique that targets service accounts in AD environments to extract and crack their password hashes. The attack exploits the way that Kerberos service tickets are encrypted and the use of weak or easily crackable passwords on service accounts. Once an attacker successfully cracks the password hashes, they can gain unauthorised access to the targeted service accounts and potentially move laterally across the network.

```
.\Rubeus.exe kerberoast
```

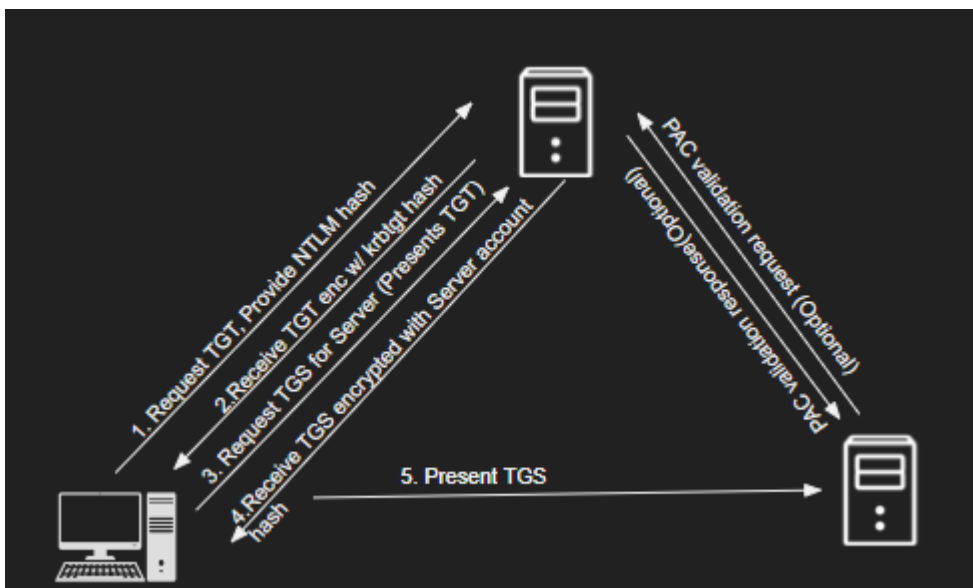
The attack works as follows:

- Identify target service accounts using SPNs (service principal names). Service accounts are often associated with services running on the network such as SQL server, Exchange or other applications.
- Request TGS Tickets: The attacker uses the identified service accounts to request TGS tickets from the KDC. These tickets contain encrypted service account password hashes.
- The attacker then attempts to crack the service account's password hash

To understand the authentication process better, consider a benign service. When a user connects to an MSSQL database using a service account with an SPN, the following steps happen:

- The client initiates the process by requesting a TGT from the KDC.

- The KDC verifies that client's identity (usually through a password hash) and issues a TGT encrypted with the krbtgt's key alongside a session key encrypted with the user's password hash. The TGT is valid for a specific period and allows the user to request service tickets without re-authenticating.
- The client sends a service ticket request (TGS-REQ) to the KDC for the MSSQL server's SPN using their TGT.
- The KDC validates the TGT and issues a TGS encrypted with the service account's secret key, containing the client's identity and a session key.
- The client connects to the MSSQL server and sends the TGS as authentication.
- The MSSQL server decrypts the TGS key using its own secret key. If the TGS is valid and the session key correct then the server accepts the connection.



We can see the above steps during network traffic analysis:

TGT	10.0.10.100	10.0.10.20	88 TCP	60 50154 → 88 [ACK] Seq=1 Ack=1 Win=262656 Len=0
	10.0.10.100	10.0.10.20	88 KRB5	349 AS-REQ
	10.0.10.20	10.0.10.100	50154 TCP	1514 88 → 50154 [ACK] Seq=1 Ack=296 Win=525568 Len=1460 [TCP segment of a reassembled PDU]
	10.0.10.20	10.0.10.100	50154 KRB5	148 AS-RFP
	10.0.10.100	10.0.10.20	88 TCP	60 50154 → 88 [ACK] Seq=296 Ack=1555 Win=262656 Len=0
	10.0.10.100	10.0.10.20	88 TCP	60 50154 → 88 [FIN, ACK] Seq=296 Ack=1555 Win=262656 Len=0
	10.0.10.20	10.0.10.100	50154 TCP	60 88 → 50154 [ACK] Seq=1555 Ack=297 Win=525568 Len=0
	10.0.10.20	10.0.10.100	50154 TCP	60 88 → 50154 [RST, ACK] Seq=1555 Ack=297 Win=0 Len=0
	10.0.10.100	10.0.10.20	88 TCP	66 50155 → 88 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM=1
	10.0.10.20	10.0.10.100	50155 TCP	66 88 → 50155 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1
TGS	10.0.10.100	10.0.10.20	88 TCP	60 50155 → 88 [ACK] Seq=1 Ack=1 Win=262656 Len=0
	10.0.10.100	10.0.10.20	88 TCP	1514 50155 → 88 [ACK] Seq=1 Ack=1 Win=262656 Len=1460 [TCP segment of a reassembled PDU]
	10.0.10.100	10.0.10.20	88 KRB5	171 TGS-REQ
	10.0.10.20	10.0.10.100	50155 TCP	60 88 → 50155 [ACK] Seq=1 Ack=1578 Win=525568 Len=0
	10.0.10.20	10.0.10.100	50155 TCP	1514 88 → 50155 [ACK] Seq=1 Ack=1578 Win=525568 Len=1460 [TCP segment of a reassembled PDU]
	10.0.10.20	10.0.10.100	50155 KRB5	117 TGS-REP
	10.0.10.100	10.0.10.20	88 TCP	60 50155 → 88 [ACK] Seq=1578 Ack=1524 Win=262656 Len=0
	10.0.10.100	10.0.10.20	88 TCP	60 50155 → 88 [FIN, ACK] Seq=1578 Ack=1524 Win=262656 Len=0
	10.0.10.20	10.0.10.100	50155 TCP	60 88 → 50155 [ACK] Seq=1524 Ack=1579 Win=525568 Len=0
	10.0.10.20	10.0.10.100	50155 TCP	60 88 → 50155 [RST, ACK] Seq=1524 Ack=1579 Win=0 Len=0
Auth	10.0.10.100	10.0.10.21	80 TCP	1514 50117 → 80 [ACK] Seq=384 Ack=1488 Win=262144 Len=1460 [TCP segment of a reassembled PDU]
	10.0.10.100	10.0.10.21	80 HTTP	1180 GET / HTTP/1.1
	10.0.10.21	10.0.10.100	50117 TCP	60 80 → 50117 [ACK] Seq=1488 Ack=2970 Win=525568 Len=0
	10.0.10.20	10.0.10.100	51306 DNS	335 Standard query response 0xe598 A download.windowsupdate.com CNAME wu-fg-shim.trafficmanager.net CNAME 2-01-3cf
	10.0.10.21	10.0.10.20	135 TCP	66 49811 → 135 [SYN, ECN, CWR] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1
	10.0.10.20	10.0.10.21	49811 TCP	66 135 → 49811 [SYN, ACK, ECN] Seq=0 Ack=1 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1
	10.0.10.21	10.0.10.20	135 TCP	60 49811 → 135 [ACK] Seq=1 Ack=1 Win=525568 Len=0
	10.0.10.21	10.0.10.20	135 DCERPC	214 Bind: call_id: 2, Fragment: Single, 3 context items: EPMV4 V3.0 (32bit NDR), EPMV4 V3.0 (64bit NDR), EPMV4 V3.0 (64bit NDR)
	10.0.10.20	10.0.10.21	49811 DCERPC	162 Bind_ack: call_id: 2, Fragment: Single, max_xmit: 5840 max_recv: 5840, 3 results: Provider rejection, Acceptance
	10.0.10.21	10.0.10.20	135 EPM	222 Map request, RPC_NETLOGON, 32bit NDR
	10.0.10.20	10.0.10.21	49811 EPM	322 Map response, RPC_NETLOGON, 32bit NDR, RPC_NETLOGON, 32bit NDR
	10.0.10.21	10.0.10.20	49669 TCP	66 49812 → 49669 [SYN, ECN, CWR] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1
	10.0.10.20	10.0.10.21	49812 TCP	66 49669 → 49812 [SYN, ACK, ECN] Seq=0 Ack=1 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1
	10.0.10.21	10.0.10.20	49669 TCP	60 49812 → 49669 [ACK] Seq=1 Ack=1 Win=525568 Len=0
	10.0.10.21	10.0.10.20	49669 DCERPC	264 Bind: call_id: 2, Fragment: Single, 3 context items: RPC_NETLOGON V1.0 (32bit NDR), RPC_NETLOGON V1.0 (64bit NDR), RPC_NETLOGON V1.0 (64bit NDR)
	10.0.10.20	10.0.10.21	49812 DCERPC	182 Bind_ack: call_id: 2, Fragment: Single, max_xmit: 5840 max_recv: 5840, 3 results: Provider rejection, Acceptance
	10.0.10.21	10.0.10.20	49669 RPC_NE	606 NetLogonSamLogonEx request
	10.0.10.20	10.0.10.21	49812 RPC_NE	190 NetLogonSamLogonEx response
	10.0.10.21	10.0.10.20	135 TCP	60 49811 → 135 [ACK] Seq=329 Ack=377 Win=525056 Len=0
	10.0.10.21	10.0.10.20	49669 TCP	60 49812 → 49669 [ACK] Seq=763 Ack=265 Win=525312 Len=0
	10.0.10.21	10.0.10.100	50117 TCP	1514 80 → 50117 [ACK] Seq=1488 Ack=2970 Win=525568 Len=1460 [TCP segment of a reassembled PDU]

During the authentication process, several security related events are generated in the windows event log:

Event ID	Event	Generates
4768	Kerberos TGT Request	When the client workstation requests the TGT from the KDC. Generates on the DC.
4769	Kerberos Service Ticket Request	After the client receives the TGT and requests a TGS.
4624	Logon	On the server, indicating a successful logon.

Since the first phase of the attack involves identifying service accounts, monitoring LDAP activity can help us in identifying the start of such an attack. An alternate approach would be to focus on the differences between benign service access and a Kerberoasting attack. In both scenarios, TGS tickets will be requested by only in the case of benign service access will the user connect to the server and present the TGS.

Detection logic could therefore entail finding all events for TGS requests and logon events from the same user and identifying where a TGS is present without a logon event. In the case of IIS service access, an additional event 4648 will be generated as a logon event.

We can now try and put all of this together to help us detect such an attack in Splunk.

```

index=main earliest=1690388417 latest=1690388630 EventCode=4648 OR
(EventCode=4769 AND service_name=iis_svc)
| dedup RecordNumber
| rex field=user "(?<username>[^\@]+)"
| table _time, ComputerName, EventCode, name, username, Account_Name,
Account_Domain, src_ip, service_name, Ticket_Options, Ticket_Encryption_Type,
Target_Server_Name, Additional_Information

```

The above query will show us benign TGS requests. We can look to see if any SPN querying has been occurring and if that is suspicious or not:

```

index=main earliest=1690448444 latest=1690454437
source="WinEventLog:SilkService-Log"
| spath input=Message
| rename XmlEventData.* as *
| table _time, ComputerName, ProcessName, DistinguishedName, SearchFilter
| search SearchFilter="*(&(samAccountType=805306368)(servicePrincipalName=*))"

```

We can then look for TGS requests where we do not have an explicit login:

```

index=main earliest=1690450374 latest=1690450483 EventCode=4648 OR
(EventCode=4769 AND service_name=iis_svc)
| dedup RecordNumber
| rex field=user "(?<username>[^\@]+)"
| bin span=2m _time
| search username!=*$
| stats values(EventCode) as Events, values(service_name) as service_name,
values(Additional_Information) as Additional_Information,
values(Target_Server_Name) as Target_Server_Name by _time, username
| where !match(Events, "4648")

```

We can do a similar thing in a slightly neater way using transactions:

```

index=main earliest=1690450374 latest=1690450483 EventCode=4648 OR
(EventCode=4769 AND service_name=iis_svc)
| dedup RecordNumber
| rex field=user "(?<username>[^\@]+)"
| search username!=*$
| transaction username keep evicted=true maxspan=5s endswith=(EventCode=4648)
startswith=(EventCode=4769)
| where closed_txn=0 AND EventCode = 4769
| table _time, EventCode, service_name, username

```

The transaction command will group events into transactions based on the username field, the `maxspan` option sets the maximum duration to 5 seconds and the `endswith` and `startswith` option specify how the transaction looks. We then filter based on transactions that are not closed and have an Event code of 4769.

AS-REPROasting

AS-REPROasting is a technique used in AD environments to target user accounts without pre-authentication enabled. Pre-auth is a feature requiring users to prove their identity before a TGT is issued, however, certain accounts do not have pre-auth enabled which makes them strong targets.

```
.\Rubeus.exe asreproast
```

The attack steps are as follows:

- The attacker identifies user accounts without pre-auth enabled.
- The attacker initiates an AS-REQ service ticket request for each identified target user account.
- The attacker captures the encrypted TGTs and employs offline password cracking to attempt to recover the user's password.

If kerberos pre-auth is enabled the AS-REQ request must contain an encrypted timestamp (`PA-ENC-TIMESTAMP`). The KDC then attempts to decrypt this timestamp using the user password hash.

11576	2021-03-06 18:35:57.601432	10.0.10.101	10.0.10.20	88	KRB5	384	AS-REQ
11578	2021-03-06 18:35:57.601868	10.0.10.20	10.0.10.101	51662	KRB5	342	AS-REP
11587	2021-03-06 18:35:57.602500	10.0.10.101	10.0.10.20	88	KRB5	392	TGS-REQ
11590	2021-03-06 18:35:57.603178	10.0.10.20	10.0.10.101	51663	KRB5	359	TGS-REP
11596	2021-03-06 18:35:57.603652	10.0.10.101	10.0.10.20	389	LDAP	582	bindRequest(3) "<ROOT>" sasl
11598	2021-03-06 18:35:57.604066	10.0.10.20	10.0.10.101	51660	LDAP	265	bindResponse(3) success
11625	2021-03-06 18:35:57.816636	10.0.10.101	10.0.10.20	5985	HTTP	88	POST /wsman/SubscriptionManager/WEC HTTP/1.

> Frame 11576: 384 bytes on wire (3072 bits), 384 bytes captured (3072 bits)

> Ethernet II, Src: VMware_f6:37:87 (00:0c:29:f6:37:87), Dst: VMware_80:c7:b1 (00:0c:29:80:c7:b1)

> Internet Protocol Version 4, Src: 10.0.10.101, Dst: 10.0.10.20

> Transmission Control Protocol, Src Port: 51662, Dst Port: 88, Seq: 1, Ack: 1, Len: 330

Kerberos

> Record Mark: 326 bytes

as-req

pvno: 5

msg-type: krb-as-req (10)

padata: 2 items

> PA-DATA pA-ENC-TIMESTAMP

> PA-DATA pA-PAC-REQUEST

req-body

Padding: 0

> kdc-options: 40810010

cname

name-type: kRB5-NT-PRINCIPAL (1)

cname-string: 1 item

CNameString: JENNY_HICKMAN

realm: LAB.INTERNAL.LOCAL

sname

till: 2037-09-13 02:48:05 (UTC)

rtime: 2037-09-13 02:48:05 (UTC)

nonce: 205710620

> etype: 6 items

addresses: 1 item ORANGE<20>

> HostAddress ORANGE<20>

If pre-auth is disabled, then this timestamp is not necessary meaning that users can request a TGT ticket without knowing the password. Kerberos authentication event 4768 contains a `PreAuthType` attribute indicating whether pre-authentication is enabled for an account. We can use this to detect AS-REProasting attacks in Splunk:

```
index=main earliest=1690392745 latest=1690393283
source="WinEventLog:SilkService-Log"
| spath input=Message
| rename XmlEventData.* as *
| table _time, ComputerName, ProcessName, DistinguishedName, SearchFilter
| search SearchFilter="*(samAccountType=805306368)
(userAccountControl:1.2.840.113556.1.4.803:=4194304)*"
```

This query looks for LDAP requests querying user accounts with Pre-Auth disabled which is a good sign of malicious activity. We can also look for TGT requests for accounts that have Pre-auth disabled:

```
index=main earliest=1690392745 latest=1690393283 source="WinEventLog:Security"
EventCode=4768 Pre_Authentication_Type=0
| rex field=src_ip "(\.:\.ffff\:)?(?:<src_ip>[0-9\.]*)"
| table _time, src_ip, user, Pre_Authentication_Type, Ticket_Options,
Ticket_Encryption_Type
```

Detecting Pass-the-Hash

Pass-the-Hash is a technique used by attackers to authenticate to a networked system using the NTLM hash of a user's password instead of the plaintext password. The attack capitalises on the way Windows stores password hashes in memory, enabling adversaries to capture the hash and reuse it for lateral movement. The attack works as follows:

- The attacker employs a tool such as Mimikatz to extract the NTLM hash of a user currently logged onto the compromised system. The attacker must have already have administrator privileges.
- The attacker uses the NTLM hash to authenticate as the targeted user on other systems or network resources without needed to know the actual password.
- Using the new session, the attacker can move laterally within the network.

An access token is a data structure that defines the security context of a process or thread. It contains information about the associated user account's identity and privileges. When a user logs on, the system verifies the password by comparing it with information stored in a security database. If the password is authenticated then the system generates an access token and any process executed on behalf of that user possesses a copy of the access token.

Alternate credentials provide a way to supply different login credentials for specific actions without altering the user's primary login sessions. This can permit a user to execute commands as another user without logging out themselves. The `runas` commands is a windows command line tool that allows users to do this. The `runas` command can be run with the `/netonly` flag which indicates that specified user information is for remote access only - this means running a `whoami` command will return the original user, but when it comes to accessing restricted folders the new cmd will be able to.

Each access token references a `LogonSession` generated at user logon which is a security structure containing information such as username, domain and `AuthenticationID`. When the `/netonly` flag is used, the process has the same access token but a different `LogonSession`.

From the windows event log perspective, the following logs are generated when the `runas` command is used:

- Event ID 4624 with `LogonType` 2 (interactive logon).
- Event ID 4624 with `LogonType` 9 (new credentials) - Only when using the `/netonly` flag.

We can aim to detect pass-the-hash attacks by Mimikatz accessing LSASS to change `LogonSession`. Then our detection can be enhanced by correlating `LogonType` 9 with Sysmon Process Access Event 10.

We can run a Splunk query to look for this:

```
index=main earliest=1690450708 latest=1690451116 source="WinEventLog:Security"
EventCode=4624 Logon_Type=9 Logon_Process=seclogo
| table _time, ComputerName, EventCode, user, Network_Account_Domain,
Network_Account_Name, Logon_Type, Logon_Process
```

As previously stated, we can enhance this search by adding LSASS memory access to our query:

```
index=main earliest=1690450689 latest=1690451116
(source="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=10
TargetImage="C:\\Windows\\system32\\lsass.exe"
SourceImage!="C:\\ProgramData\\Microsoft\\Windows
Defender\\platform\\*\\MsMpEng.exe") OR (source="WinEventLog:Security"
EventCode=4624 Logon_Type=9 Logon_Process=seclogo)
| sort _time, RecordNumber
| transaction host maxspan=1m endswith=(EventCode=4624) startswith=
(EventCode=10)
| stats count by _time, Computer, SourceImage, SourceProcessId,
Network_Account_Domain, Network_Account_Name, Logon_Type, Logon_Process
| fields - count
```

In the Sysmon part of the query we filter out LSASS memory access from windows defender to reduce the false positives. We also group related events by the host field with a maximum time of 1 minute between start and end events. We record the events that start with LSASS access and end with a successful logon.

Detecting Pass-the-Ticket

Pass-the-Ticket is a lateral movement technique used by attackers to move laterally within a network by abusing Kerberos TGT and TGS tickets. Instead of NTLM hashes, PtT leverages kerberos tickets to authenticate to other systems or networks without needing to know the user's passwords. The attack works as follows:

- The attacker gains administrative access to a system.
- The attacker uses tools such as Mimikatz or Rubeus to extract valid TGT or TGS tickets from the compromised system's memory.
- The attacker submits the extracted ticket for the current logon session.

```
.\Rubeus.exe monitor /interval:30
```



```
.\Rubeus.exe ptt /ticket:<ticket>
```

```
klist
```

Kerberos is a network authentication protocol to securely authenticate users and services within an AD environment. The related security events during access to network resources are:

Event ID	Event	Generated
4648	Explicit Credential Logon Attempt	When explicit credentials are provided during logon.
4624	Logon	When a user successfully logs on.
4672	Special Logon	When a user's logon includes special privileges.
4768	Kerberos TGT Request	When a client requests a TGT from the KDC.
4769	Kerberos Service Ticket Request	When a client requests a TGS ticket.

Detecting Pass-the-Ticket attacks can be difficult, since attackers are leveraging valid Kerberos tickets. The main difference is that when a Pass-the-Ticket attack is executed, the authentication process is only partially completed. This approach can be converted into the following Splunk detection: Look for event 4769 or 4770 without a prior 4768. Another approach would be to look for mismatches between Service and Host IDs and the actual source and destination IPs.

```
index=main earliest=1690392405 latest=1690451745 source="WinEventLog:Security"
user!=*$ EventCode IN (4768,4769,4770)
| rex field=user "(?<username>[^\@]+)"
| rex field=src_ip "(\:\:ffff\:)?(?<src_ip_4>[0-9\.]+)"
| transaction username, src_ip_4 maxspan=10h keep evicted=true startswith=
(EventCode=4768)
| where closed_txn=0
| search NOT user="*$@*"
| table _time, ComputerName, username, src_ip_4, service_name, category
```

This search looks for events from the main index within a specific time frame. These events must not be from a service account and the event code must be related to Kerberos authentication. We group events into transactions based on the `username` and `src_ip_4` fields. We then only look at open transactions that do not have a closing event.

Detecting Overpass-the-Hash

Adversaries may utilise the Overpass-the-Hash technique to obtain Kerberos TGTs by leveraging stolen password hashes to move laterally within an environment or to bypass typical system access controls. Overpass-the-Hash (A.K.A Pass-the-Key) allows authentication to occur via Kerberos rather than NTLM. Both NTLM hashes or AES keys can serve as a basis for requesting a Kerberos TGT. The attack works as follows:

- The attacker employs tools such as Mimikatz to extract the NTLM hash of a user who is currently logged in to the compromised system. The attacker must have at least local admin privileges.
- The attacker uses a tool such as Rubeus to craft a raw AS-REQ request for a specific user to request a TGT ticket. This step does not require elevated privileges on the host which makes it a stealthier approach.
- The attacker submits the requested ticket for the current logon session.

```
.\Rubeus.exe asktgt /user:<user> /domain:<domain> /rc4:<hash> ptt
```

Overpass-the-Hash leaves the same artefacts as the Pass-the-Hash attack and can be detected using the same strategies. It can be more complicated to detect however, since Rubeus sends the request directly to the DC - we should be on the lookout for unusual process communicating with the DC.

```
index=main earliest=1690443407 latest=1690443544
source="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" (EventCode=3
dest_port=88 Image!=*lsass.exe) OR EventCode=1
| eventstats values(process) as process by process_id
| where EventCode=3
| stats count by _time, Computer, dest_ip, dest_port, Image, process
| fields - count
```

This query simply looks for network communications to the DC on the Kerberos port. We can then hunt for suspicious processes or command lines.

Detecting Golden Tickets/Silver Tickets

Golden Ticket

A golden ticket attack is a potent method where an attacker forges a TGS to gain unauthorised access to a AD domain as a domain admin. The attacker creates a TGT with arbitrary user

credentials and then uses this forged ticket to impersonate a domain administrator, thereby gaining full control over the domain. A golden ticket attack is stealthy and persistent. The attack works like this:

- The attacker extracts the NTLM hash of the KRBtgt account using a DCSync attack or extracting NTDS.dit.
- Using the KRBtgt hash, the attacker forges a TGT for an arbitrary user account, and assigns it domain administrator privileges.
- The attacker injects the forged ticket in the same manner as a Pass-the-Ticket attack

```
kerberos::golden /domain:<domain> /sid:<user_sid> /rc4:<hash> /user:<nwe_user> /ptt
```

Detecting golden ticket attacks can be difficult since the TGT can be forged offline by an attacker, leaving very little trace on the system. One option would be to monitor common methods of extracting the KRBtgt hash. Alternatively, a golden ticket is just another ticket that we can detect using Pass-the-Ticket methods. In Splunk, this looks like:

```
index=main earliest=1690451977 latest=1690452262 source="WinEventLog:Security"
user!=*$ EventCode IN (4768,4769,4770)
| rex field=user "(?<username>[^\s]+)"
| rex field=src_ip "(\:\:\:\:\:\:)?(?<src_ip_4>[0-9\.\.]+)"
| transaction username, src_ip_4 maxspan=10h keep evicted=true startswith=
(EventCode=4768)
| where closed_txn=0
| search NOT user="*$@*"
| table _time, ComputerName, username, src_ip_4, service_name, category
```

Silver Ticket

Adversaries who possess the password hash of a target service account, may forge TGS tickets known as silver tickets. Silver tickets can be used to impersonate any user but are more limited in scope than golden tickets since they only allow adversaries to access a specific resource. The attack steps are:

- The attacker extracts the NTLM hash of the targeted service account using tools like Mimikatz.
- The attacker uses the extracted NTLM hash to create a forged TGS ticket for the specified service.
- The attacker injects the forged TGS in the same manner as a Pass-the-Ticket attack.

Detecting silver ticket attacks can also be difficult as there are no simple indicators of an attack. If the attackers create new user accounts you can monitor this by event id 4720. We can also look for 4762, Special logon since users might be granted privileges they shouldn't have. In Splunk we can create a csv list of users by looking at event id 4720:

```
index=main latest=1690448444 EventCode=4720
| stats min(_time) as _time, values(EventCode) as EventCode by user
| outputlookup users.csv
```

We can then compare the list with logged-in users:

```
index=main latest=1690545656 EventCode=4624
| stats min(_time) as firstTime, values(ComputerName) as ComputerName,
values(EventCode) as EventCode by user
| eval last24h = 1690451977
| where firstTime > last24h
```| eval last24h=relative_time(now(),"-24h@h")```
| convert ctime(firstTime)
| convert ctime(last24h)
| lookup users.csv user as user OUTPUT EventCode as Events
| where isnull(Events)
```

We can also look for special privileges assigned at logon:

```
index=main latest=1690545656 EventCode=4672
| stats min(_time) as firstTime, values(ComputerName) as ComputerName by
Account_Name
| eval last24h = 1690451977
```| eval last24h=relative_time(now(),"-24h@h")```
| where firstTime > last24h
| table firstTime, ComputerName, Account_Name
| convert ctime(firstTime)
```

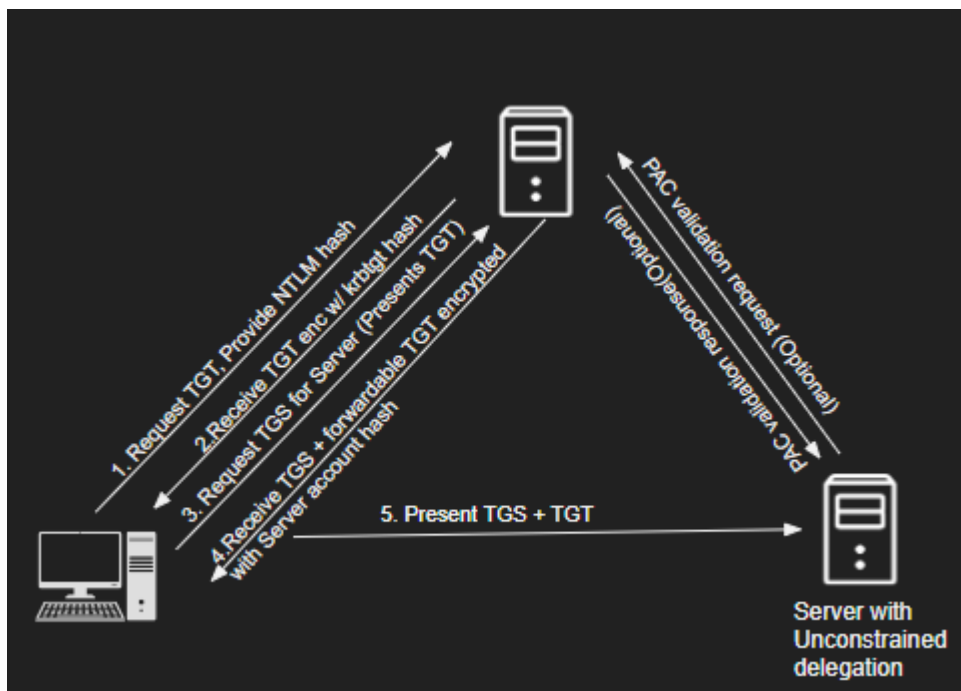
Detecting Unconstrained Delegation/Constrained Delegation Attacks

Unconstrained Delegation is a privilege that can be granted to User Accounts or Computer Accounts in an AD environment that allows a service to authenticate to another resource on behalf of any user. This feature might be necessary when a web server requires access to a database server to make changes on a user's behalf. Attack steps may be:

- The attacker identifies systems on which Unconstrained delegation is enabled for service accounts.
- The attacker gains access to a system with Unconstrained delegation enabled.
- The attacker extracts TGT tickets from the memory of the system using tools such as Mimikatz.

```
Get-ADComputer -Filter {TrustedForDelegation -eq $true -and primarygroupid -eq 515} -Properties trustedfordelgation,serviceprincipalname,description
```

When unconstrained delegation is enabled, the main difference in kerberos authentication is that when a user requests a TGS ticket for a remote service, the domain controller will embed the user's TGT into the service ticket. When connecting to the remote service, the user will present not only the TGS ticket but also their own TGT. When the service needs to authenticate to another service on behalf of the user, it will present the user's TGT ticket.



PowerShell commands and LDAP search filters used for Unconstrained Delegation discovery can be detected by monitoring powershell script block logging (event id 4104) and LDAP request logging. We can do this in Splunk:

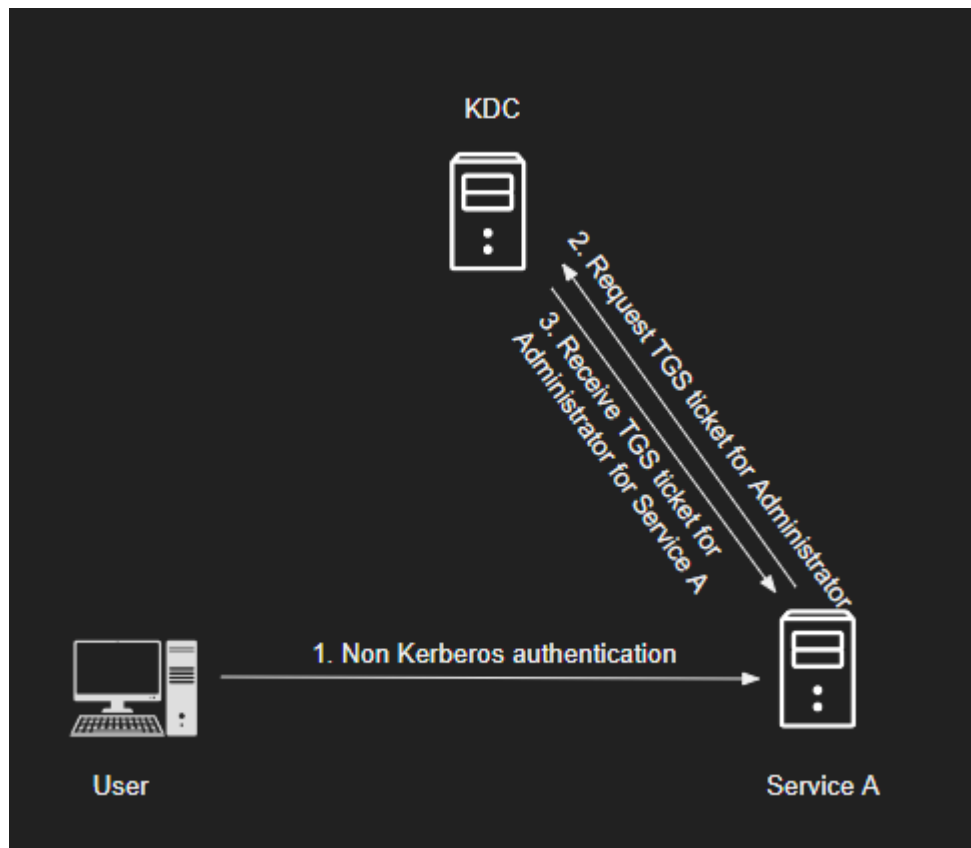
```
index=main earliest=1690544538 latest=1690544540 source="WinEventLog:Microsoft-Windows-PowerShell/Operational" EventCode=4104 Message="*TrustedForDelegation*"
OR Message="*userAccountControl:1.2.840.113556.1.4.803:=524288*"
| table _time, ComputerName, EventCode, Message
```

Constrained Delegation

Constrained delegation is a feature that allows services to delegate user credentials only to specified resources, reducing the risk associated with Unconstrained delegation. Any user or computer accounts that have SPNs set in their `msDS-AllowedToDelegateTo` property can impersonate any user in the domain to those specific SPNs. The attack steps are:

- Identify system where constrained delegation is enabled and determine the resources they are allowed to delegate.
- Gain access to the TGT of the principal user or computer. The TGT can be extracted from memory or requested with the principal's hash.
- The attacker uses the S4U technique to impersonate a high-privileged account to the targeted service.
- The attacker injects the requested ticket and accesses targeted services as the impersonated user.

S4U2self and S4U2proxy are extensions of the kerberos protocol that allow a service to request a ticket from the KDC on behalf of a user. S4U2self was designed to enable a user to request a TGS ticket when another method of authentication was used instead of kerberos. This TGS ticket can be requested on behalf of any user.



With a combination of S4U2self and S4U2proxy, an attacker can impersonate any user to SPNs set in `msDS-AllowedToDelegateTo` properties. Similar to unconstrained delegation, it is possible to detect the powershell commands and LDAP queries used to find vulnerable users and computers.

```
index=main earliest=1690544553 latest=1690562556 source="WinEventLog:Microsoft-Windows-PowerShell/Operational" EventCode=4104 Message="*msDS-AllowedToDelegateTo*"
| table _time, ComputerName, EventCode, Message
```

```
index=main earliest=1690562367 latest=1690562556
source="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational"
| eventstats values(process) as process by process_id
| where EventCode=3 AND dest_port=88
| table _time, Computer, dest_ip, dest_port, Image, process
```

Detecting DCSync/DCShadow

DCSync

DCSync is a technique exploited by attackers to extract password hashes from AD DCs. This method capitalises on the Replication Directory Changes permission typically granted to domain controllers, allowing them to read all object attributes including password hashes. Members of the Administrators, Domain admins, Enterprise admins or computer accounts on the DC have the capability to execute DCSync to extract password data from AD. This data may encompass both current and historical hashes of potentially valuable accounts such as KRBTGT and Administrators.

- The attacker secures administrative access to a domain-joined system or otherwise escalates privileges to get the prerequisite rights to request replication data.
- Using tools such as Mimikatz, the attacker requests domain replication data by using the `DRSGetNCChanges` interface - mimicking a legitimate DC.
- The attacker can use these hashes to craft golden tickets or employ PtH techniques.

`DS-Replication-Get-Changes` operations are recorded with event ID 4662. However an additional audit policy is needed since it is not enabled by default. We can seek out events with the property: `{1131f6aa-9c07-11d1-f79f-00c04fc2dcd2}` . Using Splunk we can run:

```
index=main earliest=1690544278 latest=1690544280 EventCode=4662
Message="*Replicating Directory Changes*"
| rex field=Message "(?P<property>Replicating Directory Changes.*)"
| table _time, user, object_file_name, Object_Server, property
```

DCShadow

DCShadow is an advanced tactic employed by attackers to enact unauthorised alterations to AD objects, encompassing the creation or modification of objects without producing standard security logs. The assault harnesses the Directory Replicator permission, customarily granted to domain controllers for replication tasks. Registration of a rogue DC necessitates the creation of new server and `nTDSDSA` objects in the configuration partition of the AD schema which demands Administrator privileges or the KRBTGT hash.

- The attackers secures administrative rights or otherwise escalates privileges to acquire the necessary rights.
- The attacker registers a rogue DC within the network, using the Directory Replicator permission and executes changes to AD objects.
- The rogue DC initiates replication with legitimate domain controllers which pushes the changes throughout the network.

To emulate a DC, DCShadow must implement specific modifications in AD:

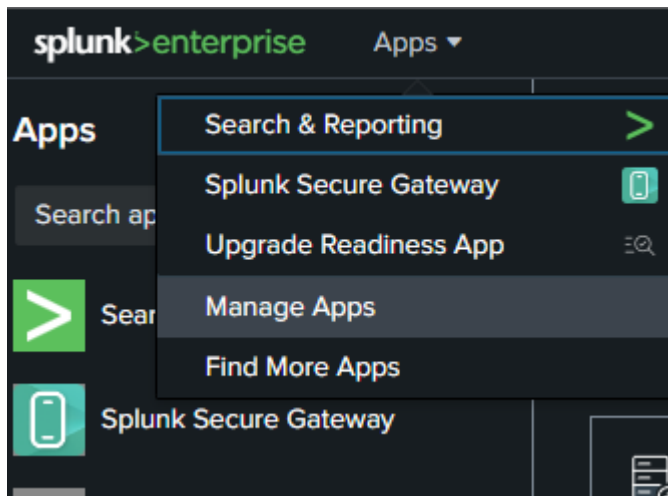
- Add a new `nTDSDSA` object.
- Append a global catalogue SPN to the computer object

Event ID 4742 logs changes related to computer objects.

```
index=main earliest=1690623888 latest=1690623890 EventCode=4742
| rex field=Message "(?P<gcspn>XX\[a-zA-Z0-9\.\-\/\]+\)"
| table _time, ComputerName, Security_ID, Account_Name, user, gcspn
| search gcspn=*
```

Creating Custom Splunk Applications

1. Access Splunk Web.
2. Go to "Manage Apps"



3. Create a new app.

4. Enter App details:

1. Name
2. Folder name
3. Version
4. Description
5. Template

Name	Academy hackthebox - Detection of Active Directory Attacks
Give your app a friendly name for display in Splunk Web.	
Folder name *	Detection_of_Active_Directory_Attacks
This name maps to the app's directory in \$SPLUNK_HOME/etc/apps/.	
Version	1.0.0
App version.	
Visible	☐ No ☒ Yes
Only apps with views should be made visible.	
Author	
Name of the app's owner.	
Description	Academy hackthebox - Detection of Active Directory Attacks
Enter a description for your app.	
Template	barebones
These templates contain example views and searches.	
Upload asset	Choose File No file chosen
Can be any html, js, or other file to add to your app.	

Cancel

Save

5. Save the app.
6. Explore the directory structure by navigating to `$SPLUNK_HOME/etc/apps` on the server. Here we will find several folders: `/bin` stores scripts; `/default` stores files for configuration, views, dashboards and app navigation; `/local` stores user-modified versions of files in the `/default` folder; `/metdata` holds permission files.
7. View the Navigation file: Using a text editor we can open the navigation configuration file: `$SPLUNK_HOME/etc/apps/<app_name>/default/data/ui/nav/default.xml`. In this file we will see the default navigation for an app.

```
<nav search_view="search">
  <view name="search" default='true' />
  <view name="analytics_workspace" />
  <view name="datasets" />
  <view name="reports" />
  <view name="alerts" />
  <view name="dashboards" />
</nav>
```

In this XML, the top-level nav tag is a parent. The `search_view` attribute designates the default view for searches. In this case, the search view is employed - inherited from the Search & Reporting app. The next level corresponds to items displayed on the app bar.

8. Create a first dashboard: Go to dashboards and click on "Create new dashboard". Enter a name and description, permissions if necessary and select "Classic dashboards"

The screenshot displays the Splunk Enterprise interface for the 'Dashboards' section. The top navigation bar includes 'Search', 'Analytics', 'Datasets', 'Reports', 'Alerts', and 'Dashboards'. A 'Create New Dashboard' button is prominently displayed. Below the button, there are 'Latest Resources' with links to 'Examples for Dashboard Studio', 'Intro to Dashboard Studio', and 'Intro to Classic Dashboards'. At the bottom, a table lists dashboards, currently showing 0. The table headers are: i, Title, Actions, Owner, App, Sharing, and Type.

Create New Dashboard



Dashboard Title

Domain Reconnaissance

domain_reconnaissance

Edit ID

Description

Optional

Permissions

Shared in App

How do you want to build your dashboard?

[What's this?](#)

Classic Dashboards

The traditional Splunk dashboard builder

Dashboard Studio

NEW

A new builder to create visually-rich, customizable dashboards

Cancel

Create

9. Configure the Dashboard

10. Dashboard storage: All dashboards created are stored at

`<app_path>/local/data/ui/views/dashboard_title.xml`. To add the dashboard to the navigation bar we add it to the navigation XML file.

11. Restart Splunk.

12. Group dashboards: If you wish to group multiple dashboards under a single entry in the nav bar we use a collection tag.

Detecting RDP Brute-Force Attacks

We will often encounter RDP brute force attacks as a vector for attackers to gain an initial foothold into a network. RDP brute-forces are very simple: the attacker attempts to log in to a remote desktop session by guessing and trying different passwords until they find the right one.

10	3.933508	192.168.152.140	192.168.152.133	TCP	74 49278 → 3389 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TSval=3802080302 TSecr=0
13	3.934508	192.168.152.133	192.168.152.140	TCP	66 3389 → 49278 [SYN, ACK] Seq=0 Ack=1 Win=64000 Len=0 MSS=1460 WS=1 SACK_PERM=1
14	3.934530	192.168.152.140	192.168.152.133	TCP	54 49278 → 3389 [ACK] Seq=1 Ack=1 Win=64256 Len=0
15	3.934696	192.168.152.140	192.168.152.133	TLShv1.2	305 Ignored Unknown Record
28	3.975777	192.168.152.133	192.168.152.140	TCP	60 3389 → 49278 [ACK] Seq=1 Ack=52 Win=63949 Len=0
33	4.170037	192.168.152.133	192.168.152.140	TLShv1.2	73 Ignored Unknown Record
34	4.170068	192.168.152.140	192.168.152.133	TCP	54 49278 → 3389 [ACK] Seq=52 Ack=20 Win=64256 Len=0
37	4.171280	192.168.152.140	192.168.152.133	TLShv1.2	371 Client Hello
39	4.172667	192.168.152.133	192.168.152.140	TLShv1.2	912 Server Hello, Certificate, Server Hello Done
40	4.172683	192.168.152.140	192.168.152.133	TCP	54 49278 → 3389 [ACK] Seq=369 Ack=878 Win=64128 Len=0
43	4.179470	192.168.152.140	192.168.152.133	TLShv1.2	372 Client Key Exchange, Change Cipher Spec, Encrypted
45	4.186056	192.168.152.133	192.168.152.140	TLShv1.2	305 Change Cipher Spec, Encrypted Handshake Message
46	4.186092	192.168.152.140	192.168.152.133	TCP	54 49278 → 3389 [ACK] Seq=687 Ack=929 Win=64128 Len=0
47	4.186589	192.168.152.140	192.168.152.133	TLShv1.2	176 Application Data
57	4.193387	192.168.152.133	192.168.152.140	TLShv1.2	347 Application Data
58	4.193402	192.168.152.140	192.168.152.133	TCP	54 49278 → 3389 [ACK] Seq=809 Ack=1222 Win=64128 Len=0
79	4.288424	192.168.152.140	192.168.152.133	TLShv1.2	662 Application Data
81	4.298786	192.168.152.133	192.168.152.140	TLShv1.2	98 Application Data
82	4.308800	192.168.152.140	192.168.152.133	TCP	54 49278 → 3389 [ACK] Seq=1437 Ack=1266 Win=64128 Len=0
91	4.388802	192.168.152.140	192.168.152.133	TLShv1.2	85 Encrypted Alert
92	4.388918	192.168.152.140	192.168.152.133	TCP	54 49278 → 3389 [FIN, ACK] Seq=1448 Ack=1266 Win=64128 Len=0
93	4.389044	192.168.152.133	192.168.152.140	TCP	60 3389 → 49278 [ACK] Seq=1266 Ack=1449 Win=62553 Len=0
104	4.406643	192.168.152.133	192.168.152.140	TCP	60 3389 → 49278 [RST, ACK] Seq=1266 Ack=1449 Win=0 Len=0

```
> Frame 15: 105 bytes on wire (840 bits), 105 bytes captured (840 bits)
> Ethernet II, Src: VMware_62:a7:9c (00:0c:29:62:a7:9c), Dst: VMware_fd:a5:e7 (00:0c:29:fd:a5:e7)
> Internet Protocol Version 4, Src: 192.168.152.140, Dst: 192.168.152.133
> Transmission Control Protocol, Src Port: 49278, Dst Port: 3389, Seq: 1, Ack: 1, Len: 51
> Transport Layer Security
```

We can use Splunk and Zeek to detect RDP brute force attacks:

Brute-force attacks are fairly straightforward to write detection logic for, because they all follow the same general steps: Attacks find services that accept credentials, they will either identify a particular user or try common default usernames and then they will try many passwords until they get the correct one.

Malware beaconing is a technique we will frequently encounter in our investigations. It refers to the periodic communication initiated by malware-infected systems with their respective C2 servers. The beacons are typically small data packets and are sent at regular intervals. The malware will use various protocols for beaconing and the timing can be fixed, jittered or follow a more complex pattern. In this section we focus on detecting the beaconing behaviour associated with Cobalt Strike in its default configuration.

```

index="cobaltstrike_beacon" sourcetype="bro:http:json"
| sort 0 _time
| streamstats current=f last(_time) as prevtime by src, dest, dest_port
| eval timedelta = _time - prevtime
| eventstats avg(timedelta) as avg, count as total by src, dest, dest_port
| eval upper=avg*1.1
| eval lower=avg*0.9
| where timedelta > lower AND timedelta < upper
| stats count, values(avg) as TimeInterval by src, dest, dest_port, total
| eval prcnt = (count/total)*100
| where prcnt > 90 AND total > 10

```

In this query we are selecting the data from a Zeek http log. We are then sorting all the data based on its timestamp in ascending order. For each event we calculate the previous events timestamp (grouped by source IP, destination IP and destination port). We then calculate the time difference between these two timestamps, calculate the average time difference across all pairs of events. We set upper and lower limit which we compare against and finally we filter the data some more to only show a proportion of the data that falls inside the limits.

```

index="cobaltstrike_beacon" sourcetype="bro:http:json"
| search id.orig_h="10.0.10.20" id.resp_h="192.168.151.181"
| timechart count

```

Detecting Nmap Port Scanning

Port scanning with Nmap is a key technique used by attackers and penetration testers all the time. We are probing networked systems for open ports that we could then exploit. When we use Nmap for port scanning, we are basically initiating a series of connection requests. We systematically attempt to establish a TCP handshake with each port in the target's address space. Nmap does not send any data to the machine other than the actual TCP handshake.

```

index="cobaltstrike_beacon" sourcetype="bro:conn:json" orig_bytes=0 dest_ip IN
(192.168.0.0/16, 172.16.0.0/12, 10.0.0.0/8)
| bin span=5m _time
| stats dc(dest_port) as num_dest_port by _time, src_ip, dest_ip
| where num_dest_port >= 3

```

In this Splunk query we are looking for empty packets that are being sent to internal IP addresses. We bin the events into 5 minute intervals based on the timestamp of each event. We then calculate the number of distinct ports that are being requested by each group of time bin,

source IP and destination IP. Finally we only display results where a machine has attempted more than 3 different empty port connections to an internal machine.

Detecting Kerberos Brute Force Attacks

When adversaries perform Kerberos-based user enumeration, they send an AS-REQ message to the KDC. This message includes the username they are trying to validate. A valid username will prompt the server to return a TGT or raise an error like `KRB5KDC_ERR_PREAUTH_REQUIRED`. On the other hand, an invalid username will be met with Kerberos error code:

`KRB5KDC_ERR_C_PRINCIPAL_UNKNOWN` in the AS-REP message. Attackers will use these responses to identify what user accounts exist on the system.

kerberos						
No.	Time	Source	Destination	Protocol	Length	Info
3206	14.697848	192.168.38.104	192.168.38.102	KRB5	287	AS-REQ
3207	14.698175	192.168.38.102	192.168.38.104	KRB5	160	KRB Error: KRB5KDC_ERR_C_PRINCIPAL_UNKNOWN
3216	14.755297	192.168.38.104	192.168.38.102	KRB5	289	AS-REQ
3217	14.755607	192.168.38.102	192.168.38.104	KRB5	160	KRB Error: KRB5KDC_ERR_C_PRINCIPAL_UNKNOWN
3226	14.811835	192.168.38.104	192.168.38.102	KRB5	291	AS-REQ
3227	14.812134	192.168.38.102	192.168.38.104	KRB5	160	KRB Error: KRB5KDC_ERR_C_PRINCIPAL_UNKNOWN
3236	14.869272	192.168.38.104	192.168.38.102	KRB5	289	AS-REQ
3237	14.869599	192.168.38.102	192.168.38.104	KRB5	160	KRB Error: KRB5KDC_ERR_C_PRINCIPAL_UNKNOWN
3246	14.926350	192.168.38.104	192.168.38.102	KRB5	292	AS-REQ
3247	14.926667	192.168.38.102	192.168.38.104	KRB5	160	KRB Error: KRB5KDC_ERR_C_PRINCIPAL_UNKNOWN
3256	14.983953	192.168.38.104	192.168.38.102	KRB5	291	AS-REQ
3257	14.984290	192.168.38.102	192.168.38.104	KRB5	160	KRB Error: KRB5KDC_ERR_C_PRINCIPAL_UNKNOWN
3266	15.042399	192.168.38.104	192.168.38.102	KRB5	288	AS-REQ
3267	15.042723	192.168.38.102	192.168.38.104	KRB5	160	KRB Error: KRB5KDC_ERR_C_PRINCIPAL_UNKNOWN
3276	15.098845	192.168.38.104	192.168.38.102	KRB5	296	AS-REQ

```
index="kerberos_bruteforce" sourcetype="bro:kerberos:json"
error_msg!=KDC_ERR_PREAUTH_REQUIRED
success="false" request_type=AS
| bin_time span=5m
| stats count dc(client) as "Unique users" values(error_msg) as "Error messages"
by _time, id.orig_h, id.resp_h
| where count>30
```

This query filters out traffic for failed Kerberos AS-REQs and removes those that are due to preauth so only the invalid user ones will remain. We then bin them based on their timestamp and calculate the distinct number of users, and returned error messages for each group based on time and source and destination IPs. Finally we filter for more than 30 users responded as we assume this is a reasonable threshold for an attack.

Detecting Kerberoasting

By possessing just one legitimate user account and its password, an attacker can retrieve SPN tickets and attempt to break them offline. After examining resources on Kerberoasting, it should be clear that RC4 is used for ticket encryption - however this can be used as a detection point.

The left screenshot shows a Zeek log entry for a Kerberos TGS-REP message. The 'kdc-options' field is expanded, showing various flags. A red box highlights the 'forwardable' and 'renewable' fields, both of which are set to 'true'. The right screenshot shows a Zeek log entry for a Kerberos TGS-REP message. The 'enc-part' field is expanded, showing the 'etype' as 'eTYPE-ARCFOUR-HMAC-MD5 (23)'. A red box highlights this field.

```
index="sharphound" sourcetype="bro:kerberos:json"
request_type=TGS cipher="rc4-hmac"
forwardable="true" renewable="true"
| table _time, id.orig_h, id.resp_h, request_type, cipher, forwardable,
renewable, client, service
```

This works because RC4 is not the modern encryption standard for kerberos tickets, but attackers will downgrade encryption to make tickets easier to crack. If we notice tickets in this configuration we should be suspicious.

```
index="sharphound" sourcetype="bro:kerberos:json"
request_type=TGS cipher="rc4-hmac"
forwardable="true" renewable="true"
| table _time, id.orig_h, id.resp_h, dest_port, request_type, cipher,
forwardable, renewable, client, service
| stats count by _time, id.orig_h, id.resp_h, dest_port, client
```

Detecting Golden Tickets

Zeek lacks the ability to accurately identify golden tickets, so instead we need to focus our search on uncovering anomalies in the ticket creation. We know that in a Golden ticket or PtT attack, the attacker bypasses the usual authentication process:

- In a Golden ticket attack, the attacker generates a forged TGT, which grants them access to any service on the network without authenticating to the KDC. Since the attacker already has a TGT, they do not need to go through the AS-REQ, AS-REP process to obtain a TGS.
- In a PtT attack, the attacker steals a valid TGT or TGS ticket from a legitimate user and then uses that ticket to access services. Again, the attacker does not need to initiate AS-

REQ, AS-REP traffic.

kerberos						
No.	Time	Source	Destination	Protocol	Length	Info
312	19.539910	192.168.38.104	192.168.38.102	KRB5	1576	TGS-REQ
314	19.542540	192.168.38.102	192.168.38.104	KRB5	1482	TGS-REP
321	19.543594	192.168.38.104	192.168.38.102	KRB5	1404	TGS-REQ
322	19.543914	192.168.38.102	192.168.38.104	KRB5	1342	TGS-REP
326	19.544609	192.168.38.104	192.168.38.102	SMB2	2974	Session Setup Request
328	19.545387	192.168.38.102	192.168.38.104	SMB2	315	Session Setup Response

```
index="golden_ticket_attack" sourcetype="bro:kerberos:json"
| where client!="-"
| bin _time span=1m
| stats values(client), values(request_type) as request_types, dc(request_type)
as unique_request_types by _time, id.orig_h, id.resp_h
| where request_types=="TGS" AND unique_request_types==1
```

This attack looks for non missing clients, bins all event into 1 minute time intervals and then calculates all client values and request types as well as the number of unique request types grouped by time and source/destination IP. The query then filters to only display events where TGS is the only request type made to the KDC.

Detecting Cobalt Strike's PSEXEC

Cobalt Strike's `psexec` command is an implementation of the PsExec tool. It is a lightweight telnet replacement that lets you execute processes on other systems. Cobalt Strike's version is optimised for executing payloads on remote systems as part of the post-exploitation process. When the `psexec` command is invoked within Cobalt Strike, the following steps occur:

- The tool creates a new service on the target system. This service is responsible for executing the desired payload. This service is typically created with a random name.
- Cobalt Strike then transfers the payload to the target system, often to the `ADMIN$` share. Typically this is done using the SMB protocol.
- After the payload has been executed, the service is stopped and deleted from the target system.
- If the payload is a beacon or backdoor, it will attempt to establish communication back to the Cobalt Strike server.

`psexec` works over port 445 and requires local administrator rights on the target system.

Protocol	Length	Info	
SMB2	655	Session Setup Request, NTLMSSP_AUTH, User: WIN10\vagrant	1
SMB2	159	Session Setup Response	
SMB2	154	Tree Connect Request Tree: \\Wef\ADMIN\$	
SMB2	138	Tree Connect Response	
SMB2	178	Ioctl Request FSCTL_QUERY_NETWORK_INTERFACE_INFO	
SMB2	474	Ioctl Response FSCTL_QUERY_NETWORK_INTERFACE_INFO	
SMB2	234	Create Request File:	
SMB2	298	Create Response File:	
SMB2	146	Close Request File:	
SMB2	182	Close Response	
SMB2	382	Create Request File: c3400ec.exe	
SMB2	410	Create Response File: c3400ec.exe	
SMB2	39474	Write Request Len:65536 Off:0 File: c3400ec.exe [TCP segment of a reassembled PDU]	
SMB2	138	Write Response	
SMB2	48138	Write Request Len:65536 Off:65536 File: c3400ec.exe	
SMB2	138	Write Response	
SMB2	1466	Write Request Len:65536 Off:131072 File: c3400ec.exe	
SMB2	138	Write Response	
SMB2	1466	Write Request Len:65536 Off:196608 File: c3400ec.exe	
SMB2	20650	Write Request Len:20480 Off:262144 File: c3400ec.exe	
SMB2	138	Write Response	
SMB2	138	Write Response	
SMB2	3242	Write Request Len:3072 Off:282624 File: c3400ec.exe	
SMB2	138	Write Response	
SMB2	162	GetInfo Request FILE_INFO/SMB2_FILE_NETWORK_OPEN_INFO File: c3400ec.exe	
SMB2	186	GetInfo Response	
SMB2	146	Close Request File: c3400ec.exe	
SMB2	182	Close Response	
SMB2	126	Tree Disconnect Request	
SMB2	126	Tree Disconnect Response	
SMB2	126	Session Logoff Request	
SMB2	126	Session Logoff Response	
SMB2	272	Negotiate Protocol Request	
SMB2	366	Negotiate Protocol Response	
SMB2	220	Session Setup Request, NTLMSSP_NEGOTIATE	
SMB2	395	Session Setup Response, Error: STATUS_MORE_PROCESSING_REQUIRED, NTLMSSP_CHALLENGE	
SMB2	655	Session Setup Request, NTLMSSP_AUTH, User: WIN10\vagrant	
SMB2	150	Session Setup Response	
SMB2	150	Tree Connect Request Tree: \\Wef\IPC\$	
SMB2	138	Tree Connect Response	
SMB2	178	Ioctl Request FSCTL_QUERY_NETWORK_INTERFACE_INFO	
SMB2	190	Create Request File: svcctl	
SMB2	474	Ioctl Response FSCTL_QUERY_NETWORK_INTERFACE_INFO	
SMB2	210	Create Response File: svcctl	
SMB2	162	GetInfo Request FILE_INFO/SMB2_FILE_STANDARD_INFO File: svcctl	
SMB2	154	GetInfo Response	
DCERPC	286	Bind: call_id: 2, Fragment: Single, 2 context items: SVCCTL V2.0 (32bit NDR), SVCCTL V2.0 (6cb11c2c-9812-4540-0300-000000000000)	
SMB2	138	Write Response	
SMB2	171	Read Request Len:1024 Off:0 File: svcctl	
DCERPC	230	Bind_ack: call_id: 2, Fragment: Single, max_xmit: 4280 max_recv: 4280, 2 results: Acceptance, Negotiate ACK	
SVCCTL	210	Unknown operation 64 request	
DCERPC	202	Fault: call_id: 2, Fragment: Single, Ctx: 0, status: nca_op_rng_error	
SVCCTL	234	OpenSCManagerW request, Wef	
SVCCTL	218	OpenSCManagerW response	
SVCCTL	330	CreateServiceA request	
SVCCTL	222	CreateServiceA response	
SVCCTL	230	StartServiceA request	
SMB2	131	Ioctl Response, Error: STATUS_PENDING	
SVCCTL	198	StartServiceA response	
SVCCTL	222	DeleteService request	
SVCCTL	198	DeleteService response	

Writing the service executable on the target via accessing hidden share "ADMIN\$"

Opening handle to svcctl to prepare for creation of new service

Service Creation

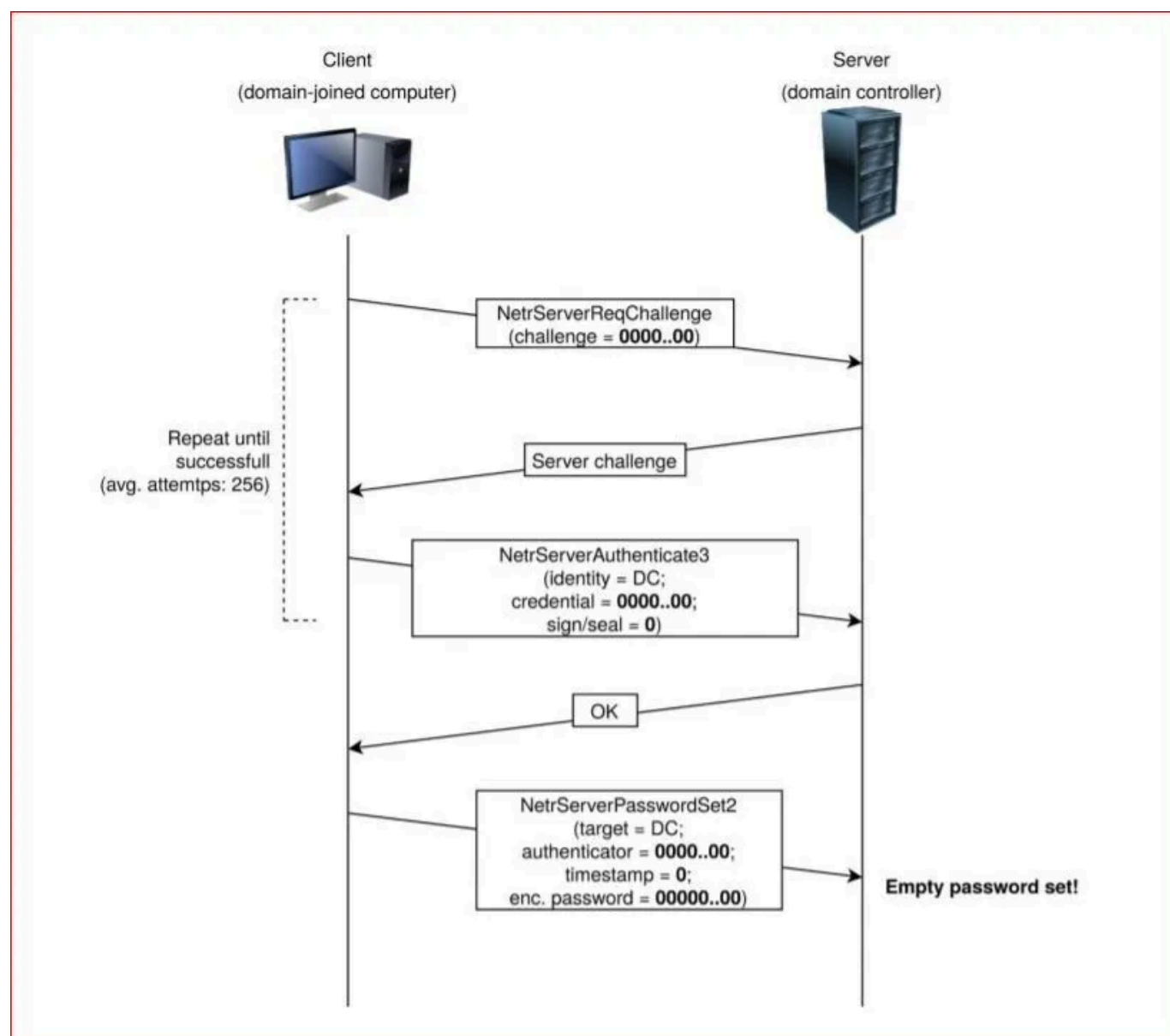
```
index="cobalt_strike_psexec"
sourcetype="bro:smb_files:json"
action="SMB::FILE_OPEN"
name IN (*.exe, *.dll, *.bat)
path IN (*.\\c$, *.\\ADMIN$)
size>0
```

Detecting Zerologon

The Zerologon vulnerability, also known as CVE-2020-1472, is a critical flaw in the implementation of the Netlogon Remote Protocol. The vulnerability can be exploited to impersonate any computer, including the DC and allow remote procedure calls on their behalf.

When a client wants to authenticate against the DC it uses a protocol called **MS-NRPC**, part of Netlogon. During this process, the client and server generate a session key computer from the machine account's password. This key is used to derive an initialisation vector for the **AES-CFB8** encryption mode. In a secure configuration these IV should be unique and random for each encryption operation. However, due to the flawed implementation of the Netlogon protocol, the IV is set to all zeros. The attacker exploits this weakness by attempting to authenticate to the DC using a session key consisting of all zeros, bypassing the authentication process. This can allow an attacker to establish a secure channel with the DC without knowing the account's password.

Once this channel is established, the attacker uses the **NetrServerPasswordSet2** function to change the computer account's password to any value essentially giving the attacker full control over the DC.



```
index="zerologon" endpoint="netlogon" sourcetype="bro:dce_rpc:json"
| bin _time span=1m
| where operation == "NttrServerReqChallenge" OR operation ==
"NttrServerAuthenticate3" OR operation == "NttrServerPasswordSet2"
| stats count values(operation) as operation_values dc(operation) as
unique_operations by _time, id.orig_h, id.resp_h
| where unique_operations >= 2 AND count>100
```

Detecting HTTP Exfiltration

Data exfiltration inside the POST body of an HTTP request is a common technique that attackers employ to extract sensitive information from a compromised system by disguising it as legitimate web traffic. It involves transmitting the stolen data to an external server operated by the attacker. Since POST requests are commonly used for a variety of legitimate purposes, this can be difficult to detect. To detect this kind of data exfiltration we need to monitor our network traffic and aggregate all data sent to specific IP addresses and ports. Analysing this data may identify patterns and anomalies that would be red flags.

```
index="cobaltstrike_exfiltration_http" sourcetype="bro:http:json" method=POST
| stats sum(request_body_len) as TotalBytes by src, dest, dest_port
| eval TotalBytes = TotalBytes/1024/1024
```

```
index="cobaltstrike_exfiltration_https" sourcetype="bro:conn:json"
(id.resp_p=443 OR id.resp_p=8080)
| stats sum(orig_bytes) as TotalBytes by src, dest, dest_port
| eval TotalBytes = TotalBytes/1024/1024
```

Detecting DNS Exfiltration

Attackers employ DNS-based exfiltration due to its reliability, stealthiness and the fact that DNS traffic is often allowed through firewalls by default. By embedding data within DNS queries and responses, attackers can bypass security controls.

- The attacker gains initial access to the victim's network.
- The attacker locates the data they want to exfiltrate and prepares it for transmission. This typically involves chunking and encoding the data.
- The attacker sends the data in the subdomains of DNS queries, using techniques such as DNS tunnelling or fast flux. They typically use a domain under their control or a

compromised domain. The attacker's DNS server receives the queries, extracts the data and reassembles it.

- After exfiltration, the attacker decodes or decrypts the data and analyses it.

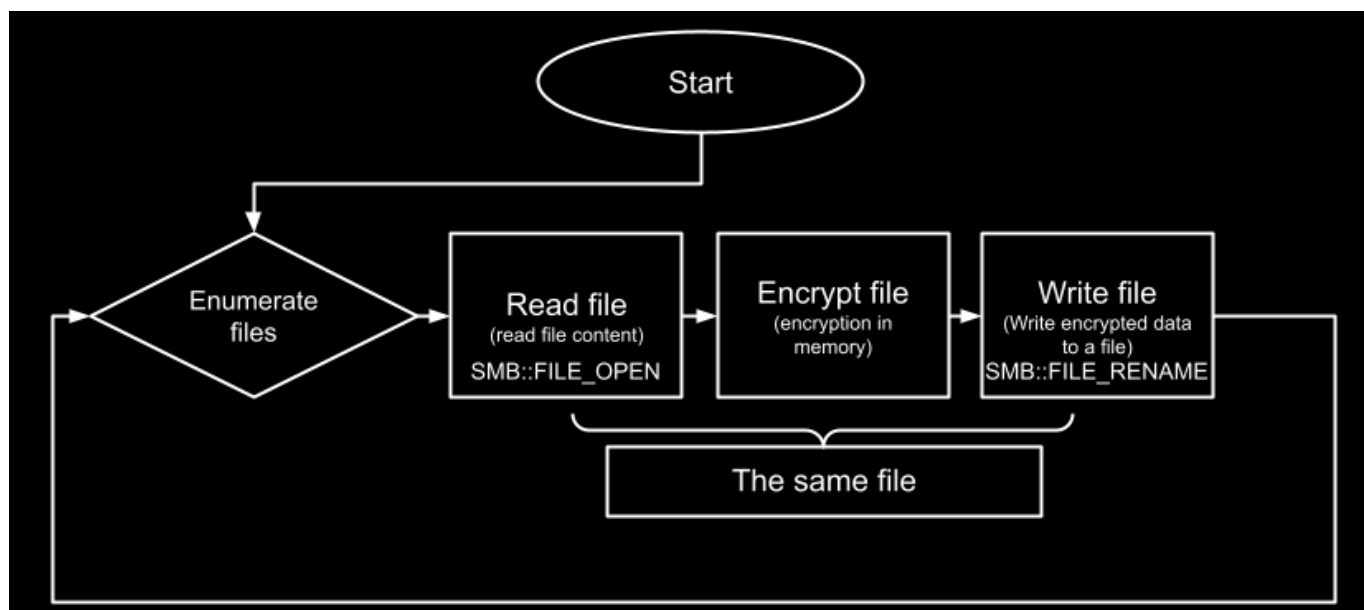
8967	196.451321	192.168.38.102	192.168.38.104	DNS	138	Standard query response 0xde2b A 111ed479a7512c9c.7c9a5671.456c54f2.blue.letsghunt.online A 0.0.0.0
8968	196.452214	192.168.38.104	192.168.38.102	DNS	122	Standard query 0x8f5a A 11483ec078e733131.8c9a5671.456c54f2.blue.letsghunt.online
8971	196.592143	192.168.38.102	192.168.38.104	DNS	138	Standard query response 0x8f5a A 11483ec078e733131.8c9a5671.456c54f2.blue.letsghunt.online A 0.0.0.0
8972	196.593094	192.168.38.104	192.168.38.102	DNS	122	Standard query 0x00b5 A 1f5e94740470d0157.9c9a5671.456c54f2.blue.letsghunt.online
8983	196.749783	192.168.38.104	192.168.38.102	DNS	122	Standard query 0x00b5 A 1f5e94740470d0157.9c9a5671.456c54f2.blue.letsghunt.online
8984	196.765666	192.168.38.102	192.168.38.104	DNS	138	Standard query response 0x00b5 A 1f5e94740470d0157.9c9a5671.456c54f2.blue.letsghunt.online A 0.0.0.0
8985	196.766564	192.168.38.104	192.168.38.102	DNS	122	Standard query 0x9942 A 114cbea690a81874a.ac9a5671.456c54f2.blue.letsghunt.online
8986	196.907655	192.168.38.102	192.168.38.104	DNS	138	Standard query response 0x9942 A 114cbea690a81874a.ac9a5671.456c54f2.blue.letsghunt.online A 0.0.0.0
8987	196.908509	192.168.38.104	192.168.38.102	DNS	122	Standard query 0x2d6c A 10db7634eade0b736.bc9a5671.456c54f2.blue.letsghunt.online
9015	197.060357	192.168.38.102	192.168.38.104	DNS	138	Standard query response 0x2d6c A 10db7634eade0b736.bc9a5671.456c54f2.blue.letsghunt.online A 0.0.0.0
9016	197.061418	192.168.38.104	192.168.38.102	DNS	122	Standard query 0x59bd A 1d5aee37e1c25ba02.cc9a5671.456c54f2.blue.letsghunt.online
9017	197.199001	192.168.38.102	192.168.38.104	DNS	138	Standard query response 0x59bd A 1d5aee37e1c25ba02.cc9a5671.456c54f2.blue.letsghunt.online A 0.0.0.0
9018	197.200130	192.168.38.104	192.168.38.102	DNS	122	Standard query 0x7809 A 1d4f517cdf8807c2.dc9a5671.456c54f2.blue.letsghunt.online
9019	197.339166	192.168.38.102	192.168.38.104	DNS	138	Standard query response 0x7809 A 1d4f517cdf8807c2.dc9a5671.456c54f2.blue.letsghunt.online A 0.0.0.0
9020	197.340089	192.168.38.104	192.168.38.102	DNS	122	Standard query 0x61f9 A 14d71477201813b75.ec9a5671.456c54f2.blue.letsghunt.online
9022	197.480990	192.168.38.102	192.168.38.104	DNS	138	Standard query response 0x61f9 A 14d71477201813b75.ec9a5671.456c54f2.blue.letsghunt.online A 0.0.0.0
9023	197.482195	192.168.38.104	192.168.38.102	DNS	122	Standard query 0xf371 A 1e3723505f4ebd907.fc9a5671.456c54f2.blue.letsghunt.online
9026	197.619489	192.168.38.102	192.168.38.104	DNS	138	Standard query response 0xf371 A 1e3723505f4ebd907.fc9a5671.456c54f2.blue.letsghunt.online A 0.0.0.0
9027	197.620375	192.168.38.104	192.168.38.102	DNS	115	Standard query 0x56c0 A 1aa645b2d.10c9a5671.456c54f2.blue.letsghunt.online
9043	197.757677	192.168.38.102	192.168.38.104	DNS	131	Standard query response 0x56c0 A 1aa645b2d.10c9a5671.456c54f2.blue.letsghunt.online A 0.0.0.0

```
index=dns_exf sourcetype="bro:dns:json"
| eval len_query=len(query)
| search len_query>=40 AND query!="*.ip6.arpa*" AND query!="*amazonaws.com*" AND
query!="*._googlecast.*" AND query!="_ldap.*"
| bin_time span=24h
| stats count(query) as req_by_day by _time, id.orig_h, id.resp_h
| where req_by_day>60
| table _time, id.orig_h, id.resp_h, req_by_day
```

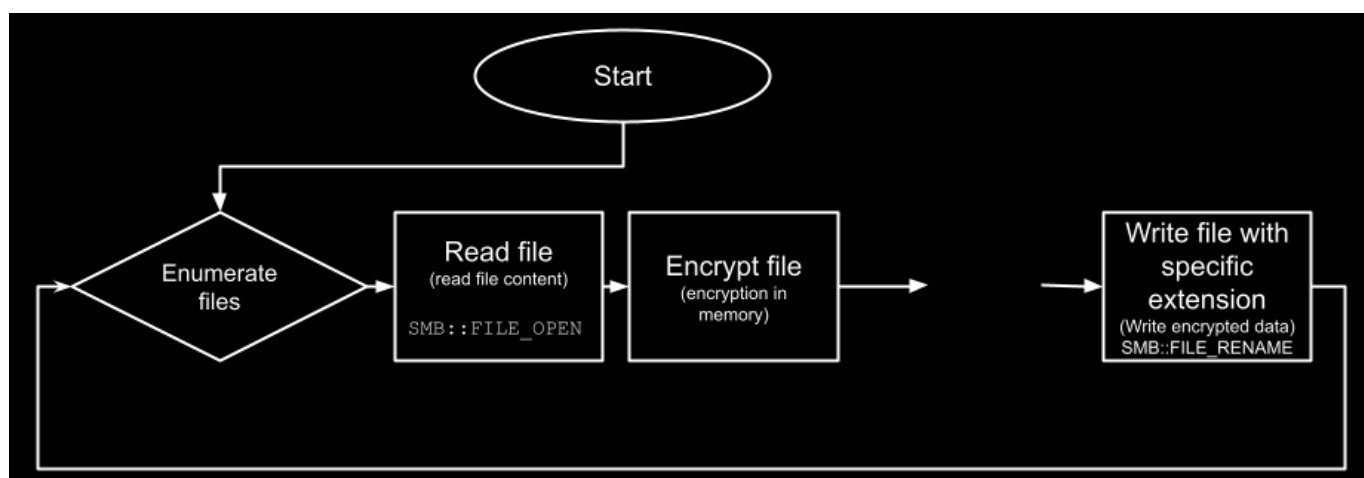
Detecting Ransomware

Ransomware leverage an array of techniques to accomplish their goals. In the following analysis, we will look at two of these methods.

1. File Overwrite Approach: Ransomware employs this tactic by accessing files through the SMB protocol, encrypting them, and then directly overwriting the original files with encrypted versions. This approach is more efficient as it requires fewer actions and leaves less traces of their activities. To detect this approach, we should look for excessive file overwrite operations.



2. File Renaming Approach: In this approach, ransomware actors use the SMB protocol to read files, they then encrypt them and finally rename the encrypted files by appending a unique extension. This signals that the files have been held hostage, making it easier for analysts and administrators to recognise an attack. Detection involves monitoring for an unusual number of files being renamed with the same extension.



```

index="ransomware_open_rename_sodinokibi" sourcetype="bro:smb_files:json"
| where action IN ("SMB::FILE_OPEN", "SMB::FILE_RENAME")
| bin _time span=5m
| stats count by _time, source, action
| where count>30
| stats sum(count) as count values(action) dc(action) as uniq_actions by _time,
source
| where uniq_actions==2 AND count>100

```

```
index="ransomware_new_file_extension_ctbl_ocker" sourcetype="bro:smb_files:json"
action="SMB::FILE_RENAME"
| bin _time span=5m
| rex field="name" "\.(?<new_file_name_extension>[^\.]*$)"
| rex field="prev_name" "\.(?<old_file_name_extension>[^\.]*$)"
| stats count by _time, id.orig_h, id.resp_p, name, source,
old_file_name_extension, new_file_name_extension,
| where new_file_name_extension!=old_file_name_extension
| stats count by _time, id.orig_h, id.resp_p, source, new_file_name_extension
| where count>20
| sort -count
```

This query searches for files that have been renamed, in particular where the old file extension is not the same as the new file extension. We group events based on time and only look at time periods where more than 20 files have been renamed within 5 minutes. Finally we sort by the count of file renames in descending order.